

Report

본 과제에서는 신용카드 사기 데이터셋을 활용해 클래스 불균형 상황에서 분류 모델을 학습하는 과정을 실습한다.

1. 데이터 로드 및 기본 탐색

개략적인 데이터 정보 및 기술통계량은 다음과 같다.

```
1) 데이터 정보 (info):
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 284807 entries, 0 to 284806
Data columns (total 31 columns):
#   Column      Non-Null Count  Dtype
---  -
0    Time      284807 non-null  float64
1    V1        284807 non-null  float64
2    V2        284807 non-null  float64
3    V3        284807 non-null  float64
4    V4        284807 non-null  float64
5    V5        284807 non-null  float64
6    V6        284807 non-null  float64
7    V7        284807 non-null  float64
8    V8        284807 non-null  float64
9    V9        284807 non-null  float64
10   V10       284807 non-null  float64
11   V11       284807 non-null  float64
12   V12       284807 non-null  float64
13   V13       284807 non-null  float64
14   V14       284807 non-null  float64
15   V15       284807 non-null  float64
16   V16       284807 non-null  float64
17   V17       284807 non-null  float64
18   V18       284807 non-null  float64
19   V19       284807 non-null  float64
20   V20       284807 non-null  float64
21   V21       284807 non-null  float64
22   V22       284807 non-null  float64
```

```
2) 기술 통계량 (describe):
```

	Time	V1	V2	V3	V4
count	284807.000000	2.848070e+05	2.848070e+05	2.848070e+05	2.848070e+05
mean	94813.859575	3.918649e-15	5.682686e-16	-8.761736e-15	2.811111e-15
std	47488.145955	1.958696e+00	1.651309e+00	1.516255e+00	1.415869e+00
min	0.000000	-5.640751e+01	-7.271573e+01	-4.832559e+01	-5.683171e+01
25%	54201.500000	-9.203734e-01	-5.985499e-01	-8.903648e-01	-8.486401e-01
50%	84692.000000	1.810880e-02	6.548556e-02	1.798463e-01	-1.984655e-01
75%	139320.500000	1.315642e+00	8.037239e-01	1.027196e+00	7.433413e-01
max	172792.000000	2.454930e+00	2.205773e+01	9.382558e+00	1.687534e+01

원본 데이터의 class 비율은 다음과 같다.

```
3) Class 비율:
0    284315
1     492
Name: Class, dtype: int64
정상 거래(0): 284315건, 사기 거래(1): 492건
```

정상 거래의 비중이 매우 높은 클래스 불균형 상황을 확인할 수 있다.

2. 샘플링

정상 거래의 비중이 너무 크므로 사기 거래는 유지하고, 정상 거래만 10000건으로 1차적인 샘플링 과정을 실행한다.

샘플링 후 클래스 비율은 다음과 같다.

샘플링 후 Class 비율:

0 10000

1 492

Name: Class, dtype: int64

정상 거래(0): 10000건, 사기 거래(1): 492건

3. 데이터 전처리

- amount가 값이 너무 크므로 표준화한다.

표준화 결과

```
memory usage: 2.0 MB
None
```

	Time	V1	V2	V3	V4	V5	
138028	82450.0	1.314539	0.590643	-0.666593	0.716564	0.301978	-1.12
63099	50554.0	-0.798672	1.185093	0.904547	0.694584	0.219041	-0.31
73411	55125.0	-0.391128	-0.245540	1.122074	-1.308725	-0.639891	0.00
164247	116572.0	-0.060302	1.065093	-0.987421	-0.029567	0.176376	-1.34
148999	90434.0	1.848433	0.373364	0.269272	3.866438	0.088062	0.97

	V7	V8	V9	...	V21	V22	V23
138028	0.388881	-0.288390	-0.132137	...	-0.170307	-0.429655	-0.141341
63099	0.495236	0.139269	-0.760214	...	0.202287	0.578699	-0.092245
73411	-0.701304	-0.027315	-2.628854	...	-0.133485	0.117403	-0.191748
164247	0.775644	0.134843	-0.149734	...	0.355576	0.907570	-0.018454
148999	-0.721945	0.235983	0.683491	...	0.103563	0.620954	0.197077

값이 작아진 것을 확인할 수 있다.

4. 학습 및 테스트 데이터 분할

학습셋 : 테스트셋 비율을 8:2로 나누고 stratify=y를 통해 클래스 비율은 유지해 준다.

```
학습 데이터 크기: (8393, 30), 테스트 데이터 크기: (2099, 30)
-----
학습 데이터 Class 비율:
0      7999
1       394
Name: Class, dtype: int64
테스트 데이터 Class 비율:
0      2001
1        98
Name: Class, dtype: int64
-----
```

학습셋 : 테스트셋 = 8:2,

각 데이터에서의 클래스 비율이 동일하게 유지됨을 확인할 수 있다.

5. SMOTE

언더샘플링을 수행했음에도 여전히 정상 거래가 사기 거래보다 약 20배 많다. 이 상태로 학습할 경우 모델은 소수 클래스인 사기 거래를 제대로 학습하지 못하고 단순히 다수 클래스로 예측하려는 경향을 보인다. 이를 해결하기 위해 SMOTE를 적용하였다.

벡터 공간 내의 선형 보간을 통해 과적합을 방지하면서 클래스 균형을 1:1로 맞춰주고자 하였다.

```
SMOTE 적용 전 학습 데이터 Class 비율:
0      7999
1       394
Name: Class, dtype: int64
-----
SMOTE 적용 후 학습 데이터 크기:
X_train_smote: (15998, 30), y_train_smote: (15998,)

SMOTE 적용 후 Class 비율:
0      7999
1      7999
Name: Class, dtype: int64
-----
```

smote 적용 후 클래스 비율이 1:1로 맞춰졌음을 확인할 수 있다.

6. ML

본 분석에서는 random forest classifier를 활용하였다. (앙상블 기법을 활용하며, 불균형 데이터에서 비교적 좋은 성능을 보임)

Classification Report:					
	precision	recall	f1-score	support	
0	0.99	1.00	1.00	2001	
1	0.95	0.89	0.92	98	
accuracy			0.99	2099	
macro avg	0.97	0.94	0.96	2099	
weighted avg	0.99	0.99	0.99	2099	
PR-AUC: 0.9537					

적용 결과는 상단과 같다.

클래스 0,1 모두에서 $\text{Recall} \geq 0.80$, $\text{F1} \geq 0.88$, $\text{PR-AUC} \geq 0.90$ 을 달성하였다.

총평

SMOTE를 활용한 데이터 증강과 Random Forest 모델의 조합을 통해 모든 정량적 목표를 초과 달성하였다. 특히 0.95 이상의 PR-AUC 점수는 모델이 사기 거래를 매우 안정적으로 구분하고 있음을 시사한다.

추가적인 조치는 다음과 같이 생각해 볼 수 있다.

1. Threshold 조정: predict_proba 임계값을 0.5가 아닌 다른 값으로 조정하여 Recall과 Precision의 Trade-off를 조절.
2. Boosting 모델 도입: XGBoost나 LightGBM과 같은 부스팅 계열 모델을 도입하고 하이퍼파라미터 튜닝을 진행하여 0.1%의 성능이라도 더 끌어올림.
3. 이상 탐지 : 지도 학습뿐만 아니라 Isolation Forest나 Autoencoder와 같은 비지도 학습 기반의 이상 탐지 기법을 앙상블하여 탐지 커버리지를 넓힘.